

Relatório 7 - Prática de estado global e integração com API

Kaíque Medeiros Lima

1. Descrição da atividade

a. Estado local, global e de servidor

A atividade trabalha a diferença entre informações que pertencem a um componente, dados compartilhados pela aplicação e conteúdo que vem de um servidor. O projeto usa Next.js e TypeScript, com Zustand para guardar o usuário de exemplo, TanStack Query para acompanhar a consulta de posts e Axios para realizar a chamada HTTP. Cada ferramenta entra em uma parte do fluxo.

O estado local fica ligado ao componente que precisa dele, como o carregamento dos formulários do card anterior. O global permite compartilhar uma informação entre partes da aplicação. Já os posts são dados externos, que podem estar carregando, falhar ou precisar de uma nova busca. Nesse projeto, os estados da consulta são fornecidos pelo TanStack Query, sem montar manualmente todos esses controles com useState.

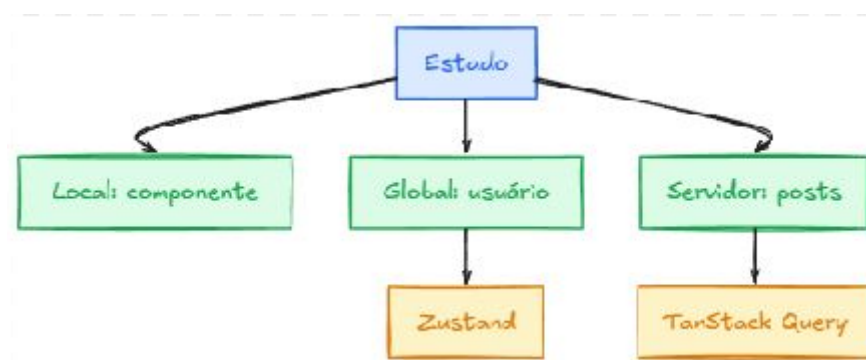


Figura 1 - Diferenças entre estado local, global e de servidor. Autoria própria, Excalidraw.

b. Usuário compartilhado com Zustand

A store useUserStore define um usuário com nome e e-mail, uma função setUser e uma ação logout. O valor inicial é null. O SiteHeader consulta essa store e mostra o botão de entrada quando não existe usuário. Ao clicar, loginMock preenche os dados de exemplo e o cabeçalho passa a apresentar uma saudação com o nome.

A ação Sair limpa o usuário e devolve a interface ao estado inicial. A store deixa essa informação disponível para outros componentes sem precisar encaminhá-la por várias props. Nessa implementação, ela não usa persistência, e a entrada é apenas simulada. Não há verificação de credenciais nem criação de uma sessão de autenticação no servidor.

c. Cliente Axios e serviço de posts

O cliente HTTP fica em data/services/api. Ele utiliza NEXT_PUBLIC_API_URL como endereço base ou, quando essa variável não está definida, o endereço do

JSONPlaceholder. Um interceptor de requisição consulta o token no localStorage e, caso exista, acrescenta o cabeçalho Authorization com o formato Bearer. Isso concentra a preparação da chamada em um só lugar.

O PostsService usa esse cliente para buscar /posts?_limit=8 e devolve response.data. A interface Post define id, title e body, descrevendo os campos usados pela tela. Essa tipagem ajuda durante o desenvolvimento, mas não valida por si só o conteúdo recebido em tempo de execução. O login mock também não grava um token; portanto, o interceptor prepara um comportamento possível, sem tornar a simulação uma autenticação real.

d. Consulta e provider do TanStack Query

O hook usePostsQuery liga a queryKey ["posts"] à função PostsService.getAll. A chave identifica essa consulta no cache, enquanto a função concentra a busca dos dados. O DashboardScreen usa o hook e recebe os posts junto com informações como isPending, isError e isFetching. Assim, a tela pode reagir ao andamento da operação sem conhecer todos os detalhes da requisição.

O ReactQueryProvider envolve a aplicação no layout principal. Dentro dele, o QueryClient é criado com a forma inicializadora do useState, mantendo a mesma instância durante a vida daquele provider. O QueryClientProvider disponibiliza esse cliente para os componentes que utilizam as consultas. Essa organização conecta o gerenciamento dos dados externos à estrutura do projeto.

CARD 07 / usePostsQuery/index.ts

```
1 import { useQuery } from "@tanstack/react-query";
2 import { PostsService } from "../../services/posts";
3
4 export const usePostsQuery = () => {
5   return useQuery({
6     queryKey: ["posts"],
7     queryFn: PostsService.getAll,
8   });
9 };
```

Figura 2 - Hook que associa a consulta de posts ao serviço. Código da branch do Card 07.

e. Carregamento, erro e atualização do dashboard

O dashboard apresenta o título e o corpo dos posts em uma lista. Enquanto a consulta inicial está pendente, aparece a mensagem Carregando posts. Se houver erro, a tela informa que não foi possível carregar os dados. Quando os posts estão disponíveis, o map monta os itens usando o id como chave.

O botão Atualizar posts chama `refetch` para solicitar uma nova busca. Durante uma requisição, `isFetching` desabilita o botão e muda seu texto para `Atualizando`. Esse cuidado deixa o usuário perceber que a ação está em andamento. O conteúdo vem da API pública, diferentemente do dashboard do card 6, que apresenta dados escritos diretamente no componente.

f. Separação das responsabilidades

O fluxo está distribuído entre a tela do dashboard, o hook de consulta, o serviço de posts e o cliente Axios. A store de usuário fica em outra área, porque representa uma informação diferente dos posts. Essa separação permite entender onde mudar o endereço da API, onde ajustar a busca e onde alterar a apresentação dos dados.

O cabeçalho e a lista também mostram que ter um usuário na store não significa proteger os dados. A consulta usa uma API pública e não depende da entrada simulada. A prática demonstra o compartilhamento de estado e o consumo de um serviço externo, mantendo visíveis os limites entre uma interface de exemplo e um sistema com autenticação completa.

2. Conclusões

O card ajuda a entender por que nem todo estado precisa ser tratado da mesma forma. Zustand organiza o usuário compartilhado, TanStack Query acompanha os dados de servidor e Axios executa a requisição. A tela usa esses resultados para mostrar carregamento, erro e conteúdo, além de permitir uma atualização manual. Com cada responsabilidade em uma parte do código, a integração fica mais fácil de acompanhar e o projeto passa a trabalhar com dados externos sem misturar toda a lógica dentro do dashboard.

3. Referências

<https://zustand.docs.pmnd.rs/>

<https://tanstack.com/query/latest/docs/framework/react/overview>

<https://axios.rest/pages/getting-started/first-steps>